

Code Explanation

Data Processing Test Code Explanation

The given code is a test suite written in Python using Pytest framework to test data processing functions. It includes tests for data cleaning and order summary table creation.

Overview of the Code

The test code is designed to validate the functionality of data processing functions, specifically `clean\_data` and `create\_order\_summary\_table`, which are imported from `src.data\_processing`. The tests utilize PySpark for data manipulation and Pytest fixtures for setting up test data.

Step 1: Setting up the SparkSession

This step creates a SparkSession, which is the entry point to any Spark functionality, using Pytest fixture. The SparkSession is configured with a specific app name, master node, and other configurations.

```
@pytest.fixture(scope="module")
def spark():
    """Create a SparkSession for testing."""
    return SparkSession.builder
        .appName("TestCustomerOrderProcessing")
        .master("local[1]")
        .config("spark.sql.warehouse.dir", tempfile.mkdtemp())
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Creating Sample Customer and Order Data

These steps create sample customer and order data using Pytest fixtures. The data includes various scenarios such as null values and duplicates.

```
@pytest.fixture(scope="module")
def sample_customer_data(spark):
    """Create sample customer data."""
    schema = StructType([
        StructField("CustId", IntegerType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    data = [
        (1, "John Doe", "john@example.com", "North"),
        (2, "Jane Smith", "jane@example.com", "South"),
        (3, "Bob Johnson", "bob@example.com", "East"),
        (4, "Alice Brown", "alice@example.com", "West"),
        (5, None, "invalid@example.com", "North"), # Null name
        (1, "John Doe", "john@example.com", "North") # Duplicate
    ]
```

```

        return spark.createDataFrame(data, schema)

@pytest.fixture(scope="module")
def sample_order_data(spark):
    """Create sample order data."""
    schema = StructType([
        StructField("OrderId", IntegerType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", IntegerType(), True)
    ])

    data = [
        (101, "Item A", 10.0, 2, datetime.date(2023, 1, 15), 1),
        (102, "Item B", 15.0, 1, datetime.date(2023, 1, 16), 2),
        (103, "Item C", 20.0, 3, datetime.date(2023, 1, 17), 3),
        (104, "Item D", 5.0, 4, datetime.date(2023, 1, 18), 4),
        (105, "Item E", 25.0, None, datetime.date(2023, 1, 19), 1),
        # Null quantity
        (101, "Item A", 10.0, 2, datetime.date(2023, 1, 15), 1) # Duplicate
    ]

    return spark.createDataFrame(data, schema)

```

Step 3: Testing the clean_data Function

This step tests the `clean\_data` function by checking if it removes null values, duplicates, and adds a new column "TotalAmount".

```

def test_clean_data(spark, sample_customer_data, sample_order_data):
    """Test the clean_data function."""
    clean_customer_df, clean_order_df = clean_data(sample_customer_data, sample_order_data)

    # Check that nulls are removed
    assert clean_customer_df.filter("Name IS NULL").count() == 0
    assert clean_order_df.filter("Qty IS NULL").count() == 0

    # Check that duplicates are removed
    assert clean_customer_df.count() == 4
    assert clean_order_df.count() == 4

    # Check that TotalAmount column is added
    assert "TotalAmount" in clean_order_df.columns

    # Check TotalAmount calculation
    row = clean_order_df.filter("OrderId = 101").first()
    assert row["TotalAmount"] == row["PricePerUnit"] * row["Qty"]

```

Step 4: Testing the create_order_summary_table Function

This step tests the `create\_order\_summary\_table` function by mocking the `save\_to\_delta` function and checking the resulting order summary table.

```

def test_create_order_summary_table(spark, sample_customer_data, sample_order_data, monkeypatch):
    """Test the create_order_summary_table function."""
    # Mock the save_to_delta function
    def mock_save_to_delta(df, catalog, schema, table):
        df.createOrReplaceTempView("ordersummary")

    # Clean the data first
    clean_customer_df, clean_order_df = clean_data(sample_customer_data, sample_order_data)

    # Create a temporary function with mocked save_to_delta
    def test_create():
        from src.data_processing import create_order_summary_table as original_func

        # Create a wrapper that uses our mock
        def wrapper(spark, customer_df, order_df):
            import types
            from src.data_processing import save_to_delta

            # Save the original function
            original_save = save_to_delta

            try:
                # Replace with our mock
                src.data_processing.save_to_delta = mock_save_to_delta

                # Call the original function
                original_func(spark, customer_df, order_df)
            finally:
                # Restore the original function
                src.data_processing.save_to_delta = original_save

        return wrapper

    # Get our test function
    test_func = test_create()

    # Run the test
    test_func(spark, clean_customer_df, clean_order_df)

    # Check the result
    result_df = spark.table("ordersummary")

    # Verify the schema
    expected_columns = ["CustId", "Name", "EmailId", "Region", "OrderId",
                        "ItemName", "PricePerUnit", "Qty", "Date",
                        "IsActive", "StartDate", "EndDate"]

    for col_name in expected_columns:
        assert col_name in result_df.columns

    # Verify the data
    assert result_df.count() == 4 # We should have 4 records after

```

```
cleaning
    assert result_df.filter("IsActive = true").count() == 4 # All s
ould be active
```